



پاسخ‌ها لزوماً یکتا نیستند و پاسخ‌های درست دیگر نیز در نظر گرفته می‌شوند.

## (۱)

### (الف)

در Planning Graph، برای تخمین هزینه رسیدن به یک مجموعه از هدف‌ها، از توابع هیوریستیک مختلفی استفاده می‌شود که معمولاً بر پایه Level Cost محاسبه می‌شوند. Level Cost برابر است با شماره اولین سطحی از گراف که آن لیترال در آن ظاهر می‌شود. سه روش پرکاربرد هیوریستیک عبارتند از:

- **Max-level Heuristic:** این روش هزینه رسیدن به کل اهداف را برابر با بیشترین Level Cost در میان تمامی لیترال‌های هدف در نظر می‌گیرد. این هیوریستیک همواره **Admissible** است، زیرا برای برآورده شدن همه اهداف، حداقل باید زمان لازم برای رسیدن به دیرترین هدف سپری شود.

- **Level-sum Heuristic:** در این روش، هزینه رسیدن به اهداف برابر با مجموع Level Cost های تک تک لیترال‌های هدف محاسبه می‌شود. این هیوریستیک معمولاً **Inadmissible** است (مگر اینکه زیرهدف‌ها کاملاً مستقل از هم باشند)، اما در عمل و برای بسیاری از مسائل سرعت همگرایی خوبی ارائه می‌دهد.

- **Set-level Heuristic:** این روش هزینه را برابر با شماره اولین سطحی در نظر می‌گیرد که در آن تمامی لیترال‌های هدف حضور داشته باشند و همزمان هیچ رابطه Mutex بین هیچ جفتی از آن‌ها وجود نداشته باشد. این هیوریستیک **Admissible** است و معمولاً تخمین دقیق‌تری نسبت به روش Max-level ارائه می‌دهد.

### (ب)

در Planning Graph، دو Action در یک سطح مشخص با یکدیگر رابطه Mutex دارند اگر نتوانند به صورت همزمان اجرا شوند. این حالت در صورت بروز حداقل یکی از سه شرط زیر رخ می‌دهد:

- **Inconsistent Effects:** زمانی اتفاق می‌افتد که اثر یک Action، با اثر دیگر در تضاد باشد. به این معنی که یکی از اعمال لیترالی را ایجاد کند (Add Effect) و عمل دیگر دقیقاً همان لیترال را خنثی یا حذف کند (Delete Effect).

- **Interference:** این حالت وقتی رخ می‌دهد که اثر یکی از اعمال، Precondition عمل دیگر را از بین ببرد. به عبارت دیگر، اجرای Action اول باعث شود شرایط لازم برای اجرای Action دوم نقض شود.

- **Competing Needs:** زمانی پیش می‌آید که Precondition‌های دو Action با یکدیگر رابطه Mutex داشته باشند. یعنی دو Action برای اجرا نیازمند Precondition‌هایی هستند که خودشان نمی‌توانند به طور همزمان در آن سطح صحیح باشند.

## (الف)

با توجه به توصیف مسئله، پهپاد و محموله در ابتدا در Base قرار دارند و گیره نگهدارنده خالی است. برای رسیدن به هدف یعنی قرار گرفتن محموله در Camp، سطوح State و Action را به صورت زیر تشکیل می‌دهیم:

- **سطح  $S_0$ :** حالت اولیه شامل لیترال‌های زیر است:  
 $At(Drone, Base), At(Cargo, Base), Empty(Drone)$
- **سطح  $A_0$ :** در این سطح Action‌های زیر به همراه NoOp‌های متناظر قابل اجرا هستند:  
 $Fly(Base, Camp)$  -  
 $Load(Cargo, Base)$  -  
 $NoOp(At(Drone, Base)), NoOp(At(Cargo, Base)), NoOp(Empty(Drone))$  -
- **سطح  $S_1$ :** لیترال‌های تولید شده از اثرات سطح قبلی عبارتند از:  
 $At(Drone, Camp)$  (از اثر Fly) -  
 $Holding(Cargo, Drone)$  (از اثر Load) -  
 $At(Drone, Base), At(Cargo, Base), Empty(Drone)$  (از NoOp‌ها) -  
 نکته مهم: در این سطح بین  $At(Drone, Camp)$  و  $Holding(Cargo, Drone)$  رابطه Mutex وجود دارد، زیرا اعمال تولیدکننده آن‌ها در  $A_0$  با یکدیگر تداخل دارند.
- **سطح  $A_1$ :** Action‌هایی که Precondition‌های آن‌ها در  $S_1$  بدون Mutex برقرار است. ترکیب کلیدی برای حل مسئله:  
 $Fly(Base, Camp)$  -  
 $NoOp(Holding(Cargo, Drone))$  -  
 (اکنون این دو Action رابطه Mutex ندارند، زیرا Fly، Precondition عمل NoOp را از بین نمی‌برد.)
- **سطح  $S_2$ :** لیترال‌های  $At(Drone, Camp)$  و  $Holding(Cargo, Drone)$  هر دو حضور دارند و دیگر با هم Mutex نیستند.
- **سطح  $A_2$ :** با توجه به برقراری Precondition‌ها بدون تضاد، Action رهاسازی اجرا می‌شود:  
 $Drop(Cargo, Camp)$  -
- **سطح  $S_3$ :** لیترال هدف یعنی  $At(Cargo, Camp)$  بدون Mutex در این سطح ظاهر شده و مسئله حل می‌شود.

## (ب)

سه مورد Mutex بین دو Action:

۱. **Inconsistent Effects**: زمانی که یک Action اثری تولید می‌کند که Action دیگر آن را Delete می‌کند.

نمونه Mutex Link:  $\text{Fly}(\text{Base}, \text{Camp})$  و  $\text{NoOp}(\text{At}(\text{Drone}, \text{Base}))$  در  $A_0$ . عمل  $\text{Fly}$  اثر  $\text{At}(\text{Drone}, \text{Base})$  دارد در حالی که  $\text{NoOp}$  لیترال  $\text{At}(\text{Drone}, \text{Base})$  را حفظ می‌کند.

۲. **Interference**: زمانی که اثر یک Action، Precondition عمل دیگر را حذف کند.  
نمونه Mutex Link:  $\text{Fly}(\text{Base}, \text{Camp})$  و  $\text{Load}(\text{Cargo}, \text{Base})$  در  $A_0$ . عمل پرواز  $\text{At}(\text{Drone}, \text{Base})$  را حذف می‌کند که دقیقاً Precondition عمل بارگیری است.

۳. **Competing Needs**: زمانی که Precondition‌های دو Action در State قبلی با یکدیگر رابطه Mutex داشته باشند.

نمونه Mutex Link: فرض کنید در  $A_1$  بخواهیم  $\text{Drop}(\text{Cargo}, \text{Camp})$  و  $\text{Load}(\text{Cargo}, \text{Base})$  را همزمان اجرا کنیم. چون Precondition‌های آن‌ها  $\text{At}(\text{Drone}, \text{Base})$  و  $\text{At}(\text{Drone}, \text{Camp})$  در  $S_1$  با هم Mutex هستند، این دو عمل نیز Mutex خواهند بود.

### دو مورد Mutex بین دو Literal:

۱. **Negated Literals**: زمانی که دو لیترال دقیقاً نقیض یکدیگر باشند (تضاد ذاتی در دامنه).  
نمونه Mutex Link:  $\text{Empty}(\text{Drone})$  و  $\text{Empty}(\text{Drone})$  - (یا جایگزین منطقی آن یعنی  $\text{Hold-}(\text{ing}(\text{Cargo}, \text{Drone}))$ ).

۲. **Inconsistent Support**: زمانی که تمام Action‌های تولیدکننده لیترال اول با تمام Action‌های تولیدکننده لیترال دوم دوه‌دو Mutex باشند.  
نمونه Mutex Link: لیترال‌های  $\text{At}(\text{Drone}, \text{Camp})$  و  $\text{Holding}(\text{Cargo}, \text{Drone})$  در سطح  $S_1$ . تنها تولیدکننده اولی در  $A_0$  عمل  $\text{Fly}$  و تنها تولیدکننده دومی عمل  $\text{Load}$  است که به دلیل Interference با هم Mutex هستند.

- **State Space**: در Forward Search، جستجو از Initial State آغاز شده و تمام اعمال ممکن اعمال می‌شوند که به یک فضای بسیار وسیع منجر می‌گردد. در Regression Search، جستجو از هدف شروع شده و تنها مسیرهایی تولید می‌شوند که پتانسیل رسیدن به هدف را دارند، بنابراین فضا متمرکزتر است.
- **Efficiency**: روش Forward Search به دلیل بررسی شاخه‌های نامربوط کارایی پایینی دارد. در مقابل، Regression Search با تمرکز بر اهداف می‌تواند سریع‌تر عمل کند، هرچند عملیات تطبیق الگو (Unification) و مدیریت متغیرها در آن سرشار محاسباتی دارد.
- **Relevance**: اعمال در Forward Search تنها به شرط برقراری Precondition اجرا می‌شوند و ممکن است Relevance پایینی نسبت به هدف داشته باشند. اما Regression Search بالاترین میزان Relevance را دارد، زیرا تنها اعمالی بررسی می‌شوند که یکی از لیترال‌های هدف را برآورده کنند.
- **Branching Factor**: در Forward Search برابر با تعداد کل اعمال مجاز در هر حالت است (بسیار بالا). در Regression Search برابر با تعداد اعمالی است که می‌توانند زیرهدف فعلی را ایجاد کنند (معمولاً بسیار کمتر).

## (ب)

- فرض کنید ۱۰۰ مکعب مختلف روی میز داریم و هدف نهایی تنها قرار دادن مکعب A روی مکعب B یعنی  $On(A, B)$  است.
- **Forward Search**: در همان قدم اول، الگوریتم می‌تواند هر کدام از ۱۰۰ مکعب را جابجا کند (Branching Factor برابر با ۱۰۰). این باعث تولید هزاران حالت بی‌ربط می‌شود.
  - **Regression Search**: الگوریتم مستقیماً بررسی می‌کند که چه عملی لیترال  $On(A, B)$  را تولید می‌کند (عمل  $Move(A, B)$ ). بنابراین بدون در نظر گرفتن ۹۸ مکعب دیگر، مسیر صحیح را شناسایی می‌کند.

## (ج)

هنگامی که Action‌ها دارای Conditional Effects (اثرات مشروط) پیچیده یا Universal Quantifiers (سورهای عمومی) باشند، اجرای این الگوریتم بسیار دشوار می‌شود. مثلاً اگر اثری به شکل «اگر شرط X برقرار بود، آنگاه Y را اضافه کن» باشد، الگوریتم برای تولید Y در حالت قبل، باید فرض کند که X برقرار بوده است. این امر باعث می‌شود حالت‌های جزئی تولید شده به سرعت پیچیده، نامشخص و حتی دارای لیترال‌های متناقض شوند که ردیابی آن‌ها رو به عقب غیرممکن یا بسیار پرهزینه است.

## (د)

در مسائلی که دارای Subgoal Interaction شدید هستند (مانند Sussman Anomaly در Blocks

(World)، برآورده کردن یک هدف باعث نقض Precondition های هدف دیگر می‌شود. از آنجا که Regression Search با مجموعه‌ای از لیترال‌های جزئی کار می‌کند، برای رفع این تداخل‌ها مجبور است تمام ترتیب‌های ممکن از اعمال (Interleaving) را حدس زده و ترکیب کند. این کار باعث انفجار در فضای جستجو می‌شود. در این حالت، Forward Search (مخصوصاً با کمک توابع Heuristic قدرتمند) بهتر عمل می‌کند، زیرا روی State های کامل و قطعی حرکت کرده و نیازی به حدس زدن تداخل‌ها ندارد.

برای یک افق زمانی مشخص  $T$ ، متغیرهای گزاره‌ای به دو دسته زیر تقسیم می‌شوند:

• **Fluents**: فرض کنید  $F$  مجموعه کل ویژگی‌های محیط باشد. برای هر  $f \in F$  و هر گام زمانی  $t$  (که  $0 \leq t \leq T$ )، یک گزاره به صورت  $f^t$  تعریف می‌شود. این گزاره زمانی درست است که ویژگی  $f$  در گام زمانی  $t$  برقرار باشد.

• **Actions**: فرض کنید  $A$  مجموعه کل اعمال ممکن باشد. برای هر  $a \in A$  و هر گام زمانی  $t$  (که  $0 \leq t \leq T - 1$ )، یک گزاره به صورت  $a^t$  تعریف می‌شود. این گزاره زمانی درست است که عمل  $a$  در گام زمانی  $t$  اجرا شود.

برای فرموله‌سازی صحیح مسئله در قالب CNF، قوانین زیر را به زبان منطق گزاره‌ای می‌نویسیم:

۱. **Preconditions & Effects**: اگر عمل  $a$  در زمان  $t$  اجرا شود، باید تمام Precondition‌های آن در زمان  $t$  برقرار بوده و تمام Effect‌های آن در زمان  $t + 1$  اعمال شوند. اگر  $\text{Pre}(a)$  مجموعه پیش‌شرط‌ها و  $\text{Eff}(a)$  مجموعه اثرات عمل باشد:

$$a^t \implies \left( \bigwedge_{p \in \text{Pre}(a)} p^t \right) \wedge \left( \bigwedge_{e \in \text{Eff}(a)} e^{t+1} \right)$$

تبدیل این رابطه به بندهای CNF به صورت زیر است:

$$\forall p \in \text{Pre}(a) : \neg a^t \vee p^t$$

$$\forall e \in \text{Eff}(a) : \neg a^t \vee e^{t+1}$$

۲. **Frame Axioms**: برای حل Frame Problem، از Successor-State Axioms استفاده می‌کنیم.

یک ویژگی  $f$  در زمان  $t + 1$  درست است اگر و تنها اگر عملی در زمان  $t$  آن را ایجاد کرده باشد (Add Effect)، و یا اینکه از قبل در زمان  $t$  درست بوده و هیچ عملی آن را حذف نکرده باشد (Delete Effect). اگر  $\text{Add}(f)$  مجموعه اعمالی باشد که  $f$  را ایجاد می‌کنند و  $\text{Del}(f)$  اعمالی باشد که  $f$  را حذف می‌کنند:

$$f^{t+1} \iff \left( \bigvee_{a \in \text{Add}(f)} a^t \right) \vee \left( f^t \wedge \neg \bigvee_{a \in \text{Del}(f)} a^t \right)$$

بندهای CNF حاصل از این رابطه دوطرفه عبارتند از:

$$\neg f^{t+1} \vee f^t \vee \bigvee_{a \in \text{Add}(f)} a^t$$

$$f^{t+1} \vee \neg f^t \vee \bigvee_{a \in \text{Del}(f)} a^t$$

۳. **Action Mutex Constraints**: دو عمل متضاد  $a_1$  و  $a_2$  که با یکدیگر تداخل دارند نباید به صورت همزمان در یک گام زمانی  $t$  اجرا شوند:

$$\neg(a_1^t \wedge a_2^t) \equiv \neg a_1^t \vee \neg a_2^t$$

(ج)

علیرغم اینکه فرمول‌های منطقی تولید شده در SATPlan بسیار بزرگ و پیچیده هستند، حل‌کننده‌های مدرن SAT به دلایل زیر سرعت و عملکرد بسیار بهتری نسبت به الگوریتم‌هایی مانند Graphplan دارند:

- **هیوریستیک‌های دینامیک** : این ابزارها بر اساس تضادهای اخیر، به صورت پویا متغیرهای کلیدی را برای ارزش‌دهی انتخاب می‌کنند، در حالی که هیوریستیک‌های Graphplan وابسته به ساختار لایه‌ای گراف و ایستا هستند.

- **ساختارهای داده بهینه مانند Two-Watched Literals**: این تکنیک به همراه بهینه‌سازی‌های سطح سخت‌افزار و حافظه پنهان (Cache)، به حل‌کننده اجازه می‌دهد میلیون‌ها بند منطقی را در هر ثانیه بررسی و فرآیند Unit Propagation را با سرعت فوق‌العاده‌ای انجام دهد.

- **دید سراسری (Global Perspective)**: الگوریتم Graphplan لایه به لایه جلو می‌رود و در هر مرحله جستجوی محلی انجام می‌دهد. اما SAT Solver کل افق زمانی  $T$  را به عنوان یک فرمول یکپارچه می‌بیند و می‌تواند تضادهای موجود بین گام‌های زمانی بسیار دور از هم را به طور همزمان کشف و رفع کند.

## ۱. تعریف گزاره‌ها (Predicates)

- $OnTable(x)$ : قطعه  $x$  روی میز قرار دارد.
- $Free(arm)$ : بازوی  $arm$  آزاد است (چیزی در دست ندارد).
- $Holding(x, arm)$ : بازوی  $arm$  قطعه  $x$  را نگه داشته است.
- $Assembled(Camera)$ : دوربین نهایی مونتاژ شده است.

## ۲. حالت اولیه (Initial State)

در ابتدا هر دو بازو آزاد هستند و هر دو قطعه روی میز قرار دارند:

$$S_0 = \{Free(LeftArm), Free(RightArm), OnTable(Lens), OnTable(Sensor)\}$$

## ۳. حالت هدف (Goal State)

هدف نهایی تنها ساخت دوربین است:

$$G = \{Assembled(Camera)\}$$

## ۴. تعریف اعمال (Actions)

- $Pickup(x, arm)$ : برداشتن قطعه  $x$  با بازوی  $arm$ 
  - Preconditions:  $Free(arm), OnTable(x)$
  - Add Effects:  $Holding(x, arm)$
  - Delete Effects:  $Free(arm), OnTable(x)$
- $Assemble(Lens, arm1, Sensor, arm2)$ : مونتاژ دو قطعه
  - Preconditions:  $Holding(Sensor, arm2), Holding(Lens, arm1)$
  - Add Effects:  $Free(arm2), Free(arm1), Assembled(Camera)$
  - Delete Effects:  $Holding(Sensor, arm2), Holding(Lens, arm1)$

۵. رسم سه سطح اول گراف برنامه‌ریزی ( $S_0 \rightarrow A_0 \rightarrow S_1$ )

• سطح  $S_0$ : شامل لیترال‌های حالت اولیه است که در بخش ۲ ذکر شد.

• سطح  $A_0$ : اعمالی که شرایط اجرایشان در  $S_0$  برقرار است:

-  $Pickup(Lens, RightArm), Pickup(Lens, LeftArm)$

- Pickup(Sensor, RightArm), Pickup(Sensor, LeftArm)

- اعمال NoOp برای تمام لیترال‌های موجود در  $S_0$ .

نکته: عمل Assemble در این سطح قابل اجرا نیست.

• **سطح  $S_1$ :** تمام اثرات مثبت (Add Effects) اعمال لایه قبل و لیترال‌های حفظ شده توسط NoOp‌ها:

- Holding(Lens, RightArm), Holding(Lens, LeftArm)

- Holding(Sensor, RightArm), Holding(Sensor, LeftArm)

- Free(RightArm), Free(LeftArm), OnTable(Sensor), OnTable(Lens)

۶. روابط **Mutex** در سطوح  $A_0$  و  $S_1$

• روابط **Mutex** در  $A_0$  (**Action Mutexes**):

- یک بازو، دو قطعه (**Interference**):

Pickup(Lens, LeftArm) و Pickup(Sensor, LeftArm) ← هر دو به Free(LeftArm)

نیاز دارند و آن را حذف می‌کنند.

Pickup(Lens, RightArm) و Pickup(Sensor, RightArm) ← تضاد روی Free(RightArm).

- یک قطعه، دو بازو (**Interference**):

Pickup(Lens, LeftArm) و Pickup(Lens, RightArm) ← هر دو به OnTable(Lens)

نیاز دارند و آن را حذف می‌کنند.

Pickup(Sensor, LeftArm) و Pickup(Sensor, RightArm) ← تضاد روی OnTable(Sensor).

• روابط **Mutex** در  $S_1$  (**Literal Mutexes**): به دلیل Inconsistent Support (چون تولیدکنندگان آن‌ها در لایه قبل Mutex بودند):

- Holding(Lens, LeftArm) و Holding(Sensor, LeftArm)

- Holding(Lens, RightArm) و Holding(Sensor, RightArm)

- Holding(Lens, LeftArm) و Holding(Lens, RightArm)

- Holding(Sensor, LeftArm) و Holding(Sensor, RightArm)

۷. بررسی امکان رسیدن به هدف در سطح  $S_1$

رسیدن به هدف در سطح  $S_1$  امکان‌پذیر نیست. لیترال هدف در این مسئله، Assembled(Camera) است. برای تولید این لیترال، ربات باید عمل Assemble را انجام دهد. از آنجا که در حالت اولیه ( $S_0$ ) ربات هیچ قطعه‌ای در دست ندارد، Precondition‌های عمل Assemble برقرار نبوده و این عمل در لایه  $A_0$  قرار نمی‌گیرد. در نتیجه، اثر آن نیز در لایه  $S_1$  ظاهر نمی‌شود. برای رسیدن به هدف، گراف باید حداقل تا لایه‌های  $A_1$  (برای اجرای Assemble) و سپس  $S_2$  (برای ظاهر شدن لیترال نهایی) گسترش یابد.